

LAPORAN PRATIUM PEMROGRAMAN ALGORITMA DAN PEMROGRAMAN
“QUEUE”



disusun Oleh:

Nama : Alhamd Ridho Putra

NIM : 2511532020

Dosen Pengampu : Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, laporan praktikum mata kuliah **Struktur Data** dengan modul "**Queue**" ini dapat terselesaikan tepat pada waktunya.

Laporan ini disusun sebagai salah satu syarat pemenuhan tugas praktikum untuk mendokumentasikan hasil implementasi dan pemahaman penulis mengenai konsep antrean dalam pemrograman. Melalui praktikum ini, penulis mendapatkan wawasan mendalam mengenai mekanisme *First-In-First-Out* (FIFO) serta penerapannya dalam struktur data yang lebih kompleks.

Penulis menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak/Ibu [**Nama Dosen Pengampu**], selaku dosen mata kuliah Struktur Data yang telah memberikan bimbingan teori.
2. Seluruh **Asisten Praktikum**, yang telah memberikan arahan teknis serta bimbingan selama pelaksanaan praktikum di laboratorium.
3. Rekan-rekan mahasiswa yang telah saling bertukar pikiran dalam menyelesaikan kendala selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penulisan maupun materi yang disampaikan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi penulis khususnya, serta bagi pembaca yang ingin memperdalam materi Struktur Data mengenai Queue.

Padang, 30 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1. Latar Belakang.....	3
1.2. Tujuan Praktikum.....	3
1.3. Manfaat Praktikum	4
BAB II PEMBAHASAN.....	5
2.1. Uraian Masalah	Error! Bookmark not defined.
2.2. Langkah Kerja	Error! Bookmark not defined.
2.3. Analisis Hasil	Error! Bookmark not defined.
BAB III KESIMPULAN.....	11
3.1. Kesimpulan	11
3.2. Saran Pengembangan	12
DAFTAR PUSTAKA.....	12

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, pengolahan data yang efisien sangat bergantung pada pemilihan struktur data yang tepat. Struktur data bukan sekadar cara menyimpan data, tetapi juga cara mengorganisasikan dan mengelola hubungan antar elemen agar operasi seperti pencarian dan manipulasi dapat dilakukan secara optimal [1]. Salah satu struktur data linear yang sangat krusial dalam berbagai implementasi sistem adalah *Queue* atau antrean.

Queue merupakan sekumpulan data yang memiliki sifat *First-In-First-Out* (FIFO), di mana elemen yang pertama kali dimasukkan akan menjadi elemen yang pertama kali dikeluarkan [2]. Prinsip ini menyerupai antrean di dunia nyata, seperti antrean pembeli di kasir atau antrean kendaraan di gerbang tol. Operasi utama dalam *Queue* dilakukan pada dua ujung yang berbeda: penambahan elemen (*enqueue*) dilakukan di ujung belakang (*rear*), sedangkan penghapusan elemen (*dequeue*) dilakukan di ujung depan (*front*).

Relevansi *Queue* dalam teknologi informasi sangat luas, mulai dari manajemen proses pada sistem operasi (*CPU Scheduling*), penanganan *buffer* pada jaringan komputer, hingga pengelolaan tugas pada *printer spooler*. Melalui praktikum ini, pemahaman mendalam mengenai implementasi *Queue* menggunakan bahasa pemrograman (seperti Java) menjadi sangat penting untuk membangun fondasi logika pemrograman yang kuat dan aplikatif.

1.2. Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah:

1. Memahami konsep dasar dan prinsip kerja struktur data *Queue* (FIFO).

2. Mampu mengimplementasikan operasi-operasi dasar *Queue* seperti *enqueue*, *dequeue*, *peek*, *isEmpty*, dan *isFull*.
3. Menganalisis perbedaan penggunaan *Queue* berbasis *array* maupun *linked list*.
4. Mampu memecahkan studi kasus sederhana yang membutuhkan logika antrian dalam penyelesaiannya.

1.3. Manfaat Praktikum

Adapun manfaat yang diharapkan dari praktikum ini adalah:

1. **Bagi Mahasiswa:** Meningkatkan kemampuan logika dalam menyusun alur program yang sistematis dan efisien, serta mempersiapkan diri untuk memahami struktur data yang lebih kompleks seperti *Trees* atau *Graphs*.
2. **Bagi Pembaca:** Memberikan referensi mengenai cara kerja dan implementasi praktis *Queue* dalam sebuah kode program.

BAB II

PEMBAHASAN

2.1. Uraian Masalah

Praktikum ini berfokus pada implementasi *Queue* (antrean) menggunakan dua pendekatan utama: *array* statis dengan logika *circular queue* dan penggunaan *Java Collections Framework*. Berikut adalah penjelasan mengenai kode program yang telah diimplementasikan:

1. **Queue Menggunakan Array (QueueArray_2511532020):** Implementasi ini menggunakan *array* dengan ukuran tetap. Masalah utama pada antrean berbasis *array* linear adalah "wasted space" saat elemen dihapus. Untuk mengatasinya, digunakan logika **Modulo (%)** pada metode enqueue dan dequeue. Operasi $(rear + 1) \% capacity$ memungkinkan *pointer* kembali ke indeks awal jika mencapai batas akhir *array*, sehingga membentuk *Circular Queue* [1].

```
package pekan4_2511532020;

public class QueueArray_2511532020 {
    int front_2020, rear_2020, size_2020;
    int capacity_2020;
    int array_2020[];

    public QueueArray_2511532020(int capacity_2020) {
        this.capacity_2020 = capacity_2020;
        front_2020 = this.size_2020 = 0;
        rear_2020 = capacity_2020 - 1;
        array_2020 = new int[this.capacity_2020];
    }

    boolean isFull(QueueArray_2511532020 queue) {
        return (queue.size_2020 == queue.capacity_2020);
    }

    boolean isEmpty(QueueArray_2511532020 queue) {
        return (queue.size_2020 == 0);
    }

    void enqueue_2511532020(int item_2020) {
        if (isFull(this))
            return;
        this.rear_2020 = (this.rear_2020 + 1) % this.capacity_2020;
        this.array_2020[this.rear_2020] = item_2020;
        this.size_2020 = this.size_2020 + 1;
        System.out.println(item_2020 + " enqueued to queue");
    }
}
```

```

    }

    int dequeue_2511532020() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        int item_2020 = this.array_2020[this.front_2020];
        this.front_2020 = (this.front_2020 + 1) %
this.capacity_2020;
        this.size_2020 = this.size_2020 - 1;
        return item_2020;
    }

    int front_2020() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;

        return this.array_2020[this.front_2020];
    }

    int rear_2020() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array_2020[this.rear_2020];
    }

    void display_2511532020() {
        int i;
        if (front_2020 == rear_2020) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }

        for (i = front_2020; i < rear_2020; i++) {
            System.out.printf(" %d <-- ", array_2020[i]);
        }
        return;
    }
}

```

```
package pekan4_2511532020;
```

```
public class QueueArrayDriver_2511532020 {
```

```

    public static void main(String[] args) {
        QueueArray_2511532020 queue = new
QueueArray_2511532020(1000);
        queue.enqueue_2511532020(10);
        queue.enqueue_2511532020(20);
        queue.enqueue_2511532020(30);
    }
}

```

```

queue.enqueue_2511532020(40);
System.out.println("item di depan " + queue.front_2020());
System.out.println("item paling belakang " + queue.rear_2020());
System.out.println("tampilan queue");

queue.display_2511532020();
System.out.println(queue.dequeue_2511532020() + " dihapus dari
queue");

System.out.println("item di depan: " + queue.front_2020());
System.out.println("item paling belakang " + queue.rear_2020());
System.out.println("tampilan queue setelah satu data di hapus");
queue.display_2511532020();

}

}

```

2. **Pembalikan Data (ReverseData_2511532020):** Kode ini menyelesaikan masalah pengubahan urutan elemen dalam *Queue*. Karena *Queue* bersifat FIFO (*First-In-First-Out*) dan *Stack* bersifat LIFO (*Last-In-First-Out*), pemindahan elemen dari *Queue* ke *Stack* dan kembali lagi ke *Queue* akan menghasilkan urutan data yang terbalik secara otomatis [2].

```

package pekan4_2511532020;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532020 {

    public static void main(String[] args) {

```



```

        Queue<Integer> q = new LinkedList<Integer>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println("sebelum reverse " + q);
        Stack<Integer> s = new Stack<Integer>();
        while (!q.isEmpty()) {
            s.push(q.remove());
        }
        while (!s.isEmpty()) {
            q.add(s.pop());
        }
        System.out.println("sesudah reverse " + q);
    }
}

```

3. **Queue Terintegrasi (QueueLinkedList & IterasiQueue):** Menggunakan antarmuka (*interface*) `java.util.Queue` dengan implementasi `LinkedList`. Bagian ini menunjukkan cara efisien menggunakan *library* standar Java untuk operasi antrean serta cara melintasi (*traversing*) data menggunakan `Iterator`.

```

package pekan4_2511532020;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532020 {

    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < 6; i++)
    }
}

```

```

        q.add(i);
        System.out.println("Elemen Antrian " + q);
        int hapus = q.remove();
        System.out.println("Hapus elemen = " + hapus);
        System.out.println(q);

        int depan = q.peek();
        System.out.println("Kepala antrian = " + depan);

        int banyak = q.size();
        System.out.println("Size Antrian = " + banyak);
    }

}

```

```

package pekan4_2511532020;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
public class IterasiQueue_2511532020 {

    public static void main(String[] args) {
        Queue<String> q = new LinkedList<>();

        q.add("Praktikum");
        q.add("Struktur");
        q.add("Data");
        q.add("Dan");
        q.add("Algoritma");
        Iterator<String> Iterator = q.iterator();
        while (Iterator.hasNext()) {

```

```

        System.out.print(Iterator.next() + " ");
    }
}
}

```

2.2. Langkah Kerja

Langkah-langkah dalam melaksanakan praktikum ini adalah sebagai berikut:

1. **Pendefinisian Struktur:** Menentukan variabel kontrol seperti front, rear, size, dan capacity.
2. **Inisialisasi Konstruktor:** Membuat objek *Queue* dengan kapasitas tertentu dan mengatur posisi awal front di 0 serta rear di kapasitas - 1.
3. **Implementasi Metode Utama:**
 - enqueue_2511532020: Menambahkan elemen ke posisi rear yang baru.
 - dequeue_2511532020: Mengambil elemen dari posisi front dan menggeser pointer front.
 - isEmpty dan isFull: Melakukan pengecekan kondisi limitasi memori.
4. **Pengujian Driver:** Menjalankan QueueArrayDriver_2511532020 untuk memvalidasi apakah data yang masuk pertama kali benar-benar keluar pertama kali.
5. **Manipulasi Data:** Menerapkan struktur *Stack* untuk membalikkan urutan elemen dalam *Queue*.
6. **Iterasi:** Menggunakan objek Iterator untuk menampilkan seluruh elemen antrian tanpa merusak struktur data di dalamnya.

2.3. Analisis Hasil

Berdasarkan hasil eksekusi program, diperoleh analisis sebagai berikut:

- **Efisiensi Operasi:** Implementasi *Queue* berbasis *array* maupun *linked list* memiliki kompleksitas waktu sebesar $O(1)$ untuk operasi penambahan (*enqueue*) dan penghapusan (*dequeue*). Hal ini dikarenakan program

langsung mengakses indeks front atau rear tanpa melakukan perulangan (*looping*) [3].

- **Mekanisme Circular:** Pada kelas `QueueArray_2511532020`, penggunaan modulo terbukti efektif menjaga penggunaan memori. Namun, perlu dicatat bahwa metode `display_2511532020` pada kode tersebut masih menggunakan perulangan linear ($i < \text{rear}$). Pada kondisi *circular* yang sebenarnya (di mana rear mungkin lebih kecil dari front), logika tampilan perlu diperbaiki agar dapat mencetak data secara melingkar.
- **Pembalikan Elemen:** Pada kelas `ReverseData_2511532020`, penggunaan *Stack* sebagai penampung sementara adalah cara paling optimal untuk membalikkan *Queue*. Output menunjukkan urutan [1, 2, 3] berubah menjadi [3, 2, 1], yang membuktikan interaksi antar struktur data linear berjalan sesuai logika.

BAB III

KESIMPULAN

3.1. Kesimpulan

Berikut adalah draf **Bab III Penutup** serta **Daftar Pustaka** lengkap yang menggabungkan seluruh referensi dari Bab I hingga Bab III dengan format IEEE.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum dan analisis yang telah dilakukan terhadap struktur data *Queue*, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Mekanisme FIFO:** *Queue* merupakan struktur data linear yang bekerja berdasarkan prinsip *First-In-First-Out* (FIFO), di mana elemen yang pertama kali masuk akan menjadi yang pertama kali diproses atau dihapus.
2. **Efisiensi Implementasi:** Implementasi *Queue* baik menggunakan *array* maupun *LinkedList* memberikan efisiensi tinggi dengan kompleksitas waktu rata-rata $O(1)$ untuk operasi *enqueue* dan *dequeue*.
3. **Optimalisasi Memori:** Penggunaan logika *Circular Queue* pada media berbasis *array* sangat penting untuk mencegah pemborosan ruang memori yang terjadi akibat pergeseran indeks penunjuk (*pointer*).
4. **Fleksibilitas Struktur Data:** Struktur data *Queue* dapat dimanipulasi menggunakan bantuan struktur data lain, seperti *Stack*, untuk melakukan operasi pembalikan urutan data secara sistematis.

3.2. Saran Pengembangan

Untuk pengembangan lebih lanjut dalam pemahaman maupun implementasi *Queue*, disarankan hal-hal sebagai berikut:

1. **Eksplorasi Variasi Queue:** Disarankan untuk mempelajari variasi lain seperti *Priority Queue*, di mana elemen keluar berdasarkan prioritas tertentu bukan sekadar urutan kedatangan [1].
2. **Implementasi Deque:** Mencoba implementasi *Double-Ended Queue* (Deque) yang memungkinkan penambahan dan penghapusan elemen dari kedua ujung (*front* dan *rear*).
3. **Visualisasi GUI:** Mengembangkan aplikasi berbasis grafis (GUI) untuk memvisualisasikan pergerakan elemen dalam antrean guna memudahkan pemahaman bagi pengguna awam.

4. **Penanganan Error:** Memperkuat pengecekan kondisi *Error Handling* pada kode program, seperti penanganan *exception* ketika melakukan *dequeue* pada antrean yang kosong agar program tidak berhenti secara paksa (*crash*).

TUGAS PRAKTIKUM

```
import java.util.Scanner;

public class AntrianLoket_2511532020 {
    // Variabel wajib sesuai aturan tugas
    int front, rear, max;
    String queue[];

    // Konstruktor untuk inisialisasi
    public AntrianLoket_2511532020(int kapasitas) {
        max = kapasitas;
        queue = new String[max];
        front = -1;
        rear = -1;
    }

    // Mengecek apakah antrian penuh
    public boolean isFull() {
        return rear == max - 1;
    }

    // Mengecek apakah antrian kosong
    public boolean isEmpty() {
        return front == -1 || front > rear;
    }

    // Menambahkan pelanggan ke belakang antrian
    public void enqueue(String nama) {
        if (isFull()) {
```

```

        System.out.println("Antrian penuh! Tidak dapat menambah
pelanggan.");
    } else {
        if (isEmpty()) {
            front = 0;
        }
        rear++;
        queue[rear] = nama;
        System.out.println("Data berhasil ditambahkan ke antrian");
    }
}

// Menghapus pelanggan dari depan antrian (FIFO)[cite: 1]
public void dequeue() {
    if (isEmpty()) {
        System.out.println("Antrian kosong! Tidak ada pelanggan untuk
dilayani.");
    } else {
        System.out.println(queue[front] + " telah dilayani");
        front++;
        // Reset jika setelah dihapus menjadi kosong
        if (front > rear) {
            front = rear = -1;
        }
    }
}

// Menampilkan seluruh isi antrian[cite: 1]
public void display() {
    if (isEmpty()) {

```



```

        System.out.println("Isi antrian: (Kosong)");
    } else {
        System.out.println("Isi antrian:");
        int nomor = 1;
        for (int i = front; i <= rear; i++) {
            System.out.println(nomor + ". " + queue[i]);
            nomor++;
        }
    }
}

// Membalik seluruh isi antrian[cite: 1]
public void Reverse() {
    if (isEmpty()) {
        System.out.println("Antrian kosong, tidak ada yang bisa dibalik.");
    } else {
        int i = front;
        int j = rear;
        while (i < j) {
            String temp = queue[i];
            queue[i] = queue[j];
            queue[j] = temp;
            i++;
            j--;
        }
        System.out.println("Antrian berhasil dibalik.");
    }
}

public static void main(String[] args) {

```

```

Scanner input = new Scanner(System.in);
// Inisialisasi kapasitas antrian, misal 10
AntrianLoket_2511532020 loket = new AntrianLoket_2511532020(10);
int pilihan;

do {
    System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
    System.out.println("1. Tambah Antrian");
    System.out.println("2. Hapus Antrian");
    System.out.println("3. Tampilkan Antrian");
    System.out.println("4. Reverse");
    System.out.println("5. Keluar");
    System.out.print("Pilih menu: ");
    pilihan = input.nextInt();
    input.nextLine(); // membersihkan buffer

    switch (pilihan) {
        case 1:
            System.out.print("Masukkan nama pelanggan: ");
            String nama = input.nextLine();
            loket.enqueue(nama);
            break;
        case 2:
            loket.dequeue();
            break;
        case 3:
            loket.display();
            break;
        case 4:
            loket.Reverse();
    }
}

```

```
        loket.display();
        break;
    case 5:
        System.out.println("Program selesai");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
} while (pilihan != 5);

input.close();
}
}
```

DAFTAR PUSTAKA

- [1] A. Tenenbaum, Y. Langsam, and M. Augenstein, *Data Structures Using C*. New Delhi: Prentice-Hall of India, 1990.
- [2] R. Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis: Sams Publishing, 2002.
- [3] M. T. Goodrich, R. Tamassia, and D. Mount, *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons, 2013.
- [4] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston: Pearson, 2012.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [6] G. F. Isaacs, "Stack-based Evaluation of Arithmetic Expressions," *Journal of Computing Sciences in Colleges*, vol. 17, no. 5, pp. 10-21, 2002.
- [7] S. Sahni, *Data Structures, Algorithms, and Applications in Java*. Silicon Press, 2004.